Calidad de Software aplicada al TFG: NexusAI

Trabajo Práctico Integrador — Informe final

Delfina Salinas Santiago Tricherri Marcos Bugliotti

Universidad Católica de Córdoba · Calidad de Software · 26 de octubre de 2026

Índice

1. Resumen **2**

2. El sistema y su nivel **2**
2.1. Qué es NexusAI . . . 2
2.2. Nivel declarado: 3, prototipo avanzado . . . 3

3. Metas de calidad **3**
3.1. Metas del nivel 3, una por una . . . 3
3.2. Lo que quedó parcial o no cumplido . . . 3

4. Los ejes del trabajo **4**
4.1. Introducción a la calidad . . . 4
4.2. Fundamentos del testing . . . 4
4.3. Modelos de calidad y niveles de prueba . . . 4
4.4. Métricas y análisis estático . . . 5
4.5. Pruebas dinámicas . . . 5
4.6. Aseguramiento de la calidad . . . 5
4.7. Automatización del testing . . . 6
4.8. Herramientas del ecosistema . . . 6

5. Evolución de las métricas **6**
5.1. Producto (SonarCloud de `development`, 15/10 → 20/10) . . . 6
5.2. Mejoras medidas antes/después . . . 7
5.3. Métrica de proceso: lead time de los PRs . . . 7
5.4. Métrica de proyecto: avance y velocidad . . . 7
5.5. Desempeño . . . 8
5.6. Ensayo de la demo (21/10) . . . 8

6. Defectos **8**

7. Análisis crítico **9**
7.1. Lo que funcionó . . . 9
7.2. Lo que aprendimos de lo que no salió bien . . . 9
7.3. Riesgos que quedan abiertos . . . 9
7.4. Qué haríamos distinto . . . 10

8. Conclusiones **10**

9. Aporte individual **10**

10. Uso de inteligencia artificial **11**

11. Anexo A: entregables del enunciado **11**

12. Anexo B: índice de documentos **11**

1. Resumen

NexusAI es un asistente académico con IA que vive dentro de Moodle. Tiene tres partes: un plugin de Moodle (`local_nexusai`), un widget en React y un backend en FastAPI con PostgreSQL/pgvector y Redis. El backend responde preguntas usando el material del curso (RAG) y un proveedor de IA externo. Por la etapa de la tesis, el proyecto corresponde al **nivel 3 (prototipo avanzado)**.

Durante el TP, entre el 13 y el 26/10/2026, aplicamos un proceso de calidad completo organizado en once subtareas (épica #584). **Se cumplen todas las metas del nivel 3:**

- pipeline con **quality gate bloqueante** (SonarCloud);
- **83,6 % de cobertura** sobre el código existente (la meta es 80 %; partimos de 56,6 %);
- **despliegue automático a staging** con verificación del commit y **smoke E2E** contra el proveedor real en cada deploy;
- integración externa probada contra el **entorno real**;
- **stress y esfuerzo sostenido** con umbrales evaluados por el pipeline.

SonarCloud pasó de **D/D/A a A/A/A** (confiabilidad, seguridad, mantenibilidad), con **26 → 0 bugs** y **14 → 0 vulnerabilidades**.

El proceso encontró **10 defectos reales**, cada uno corregido con su prueba de regresión. El más importante lo encontró la prueba de stress: con 40 alumnos simultáneos el chat **colapsaba**, por un interbloqueo en el pool de conexiones a la base. Después del fix, el sistema atiende **100 alumnos simultáneos con un p95 de 2,5 s**.

Lo que no cumplimos o quedó parcial se detalla en la sección 3.2: Behat sin escenarios, tres brechas de trazabilidad y la prueba diaria contra el proveedor, que recién corre desde `main`. Todos los números de este informe salen de corridas del pipeline o de la API de SonarCloud; cada uno tiene su link.

2. El sistema y su nivel

2.1. Qué es NexusAI

Componente	Tecnología	Qué hace
Plugin `local_nexusai`	PHP 8.1–8.4, Moodle 4.5–5.3	Integra el asistente en el curso, calcula qué material ve cada alumno y firma cada pedido al backend (HMAC)
Widget	React 18, compilado a módulos AMD	Chat, búsqueda, quiz, resúmenes y gestión de documentos
Backend	FastAPI, PostgreSQL + pgvector, Redis	RAG sobre el material del curso, límites de uso, registro de consumo

Integraciones externas (el centro del TP, según el enunciado):

- **el proveedor de IA**: Gemini y OpenAI, con API compatible con OpenAI, para chat y embeddings, con una cadena de fallback;
- **Moodle**: web services, Privacy API y eventos;
- **la nube de Oracle**: las VMs de staging y producción.

2.2. Nivel declarado: 3, prototipo avanzado

El profesor aclaró el 15/10 que el nivel lo define la etapa de la tesis: nivel 3 corresponde a *"la mayoría implementada, el flujo principal funciona con integraciones externas y se puede desplegar"*. NexusAI cumple las tres condiciones: está en producción en una VM de Oracle, y el plugin pasó la auditoría previa para publicarse en el Moodle Plugins Directory (versión 0.19.0). Se trabajó contra las metas del nivel 3 desde el primer día (LINEA_BASE.md).

3. Metas de calidad

3.1. Metas del nivel 3, una por una

Aspecto	Meta	Resultado	Evidencia
Pipeline	Compilación, pruebas, análisis estático, cobertura, quality gate y reporte automático	**Cumplida**	`backend-ci`, `frontend-ci`, `moodle-ci` (7 combinaciones de Moodle/PHP/BD), `quality-gate`
Quality gate	Bloquea bugs críticos, duplicación > 5 % y cobertura de código nuevo < 80 %	**Cumplida** (más estricto)	SonarCloud "Sonar way" + protección de `development`. Demostrado en rojo en el PR #602 (cobertura 0 %, duplicación 72,3 %)
Cobertura de código nuevo	80 %	**Cumplida**	Condición del gate en cada PR; por ejemplo, 83,1 % en #646
Cobertura de código existente	**80 %**	**Cumplida: 83,6 %**	Corrida 38819389914 (10.812 de 12.931 líneas)
Plan de pruebas y riesgos	Versionado, con criterios de entrada/salida, riesgos P×I y trazabilidad	**Cumplida**	`PLAN_DE_PRUEBAS.md` v1.3 · `TRAZABILIDAD.md`
Defectos	Registrados con pasos, esperado y obtenido, vinculados a su caso	**Cumplida**, en GitHub Issues	Formulario "Defecto", labels de severidad, `REGISTRO_DEFECTOS.md`
Herramienta nueva	Proceso de cinco pasos	**Cumplida**	`HERRAMIENTA_SONARCLOUD.md` (requisitos, mercado, prueba de concepto, matriz, despliegue)
Smoke test	Ejecutado por el pipeline después del deploy	**Cumplida**	`deploy-staging.yml`: verde y rojo provocado
Integración externa	**Flujo E2E automatizado en cada despliegue**	**Cumplida**	El smoke *es* el E2E: chat firmado con HMAC → base → embeddings → LLM real → anti-replay
Stress y esfuerzo sostenido	Ejecutados, con resultados y umbrales	**Cumplida**	`load-test.yml` (k6), corrida oficial 39330079364

3.2. Lo que quedó parcial o no cumplido

Lo informamos sin maquillarlo:

1. **Registro en Jira.** El enunciado dice “Jira u otra herramienta”. Usamos **GitHub Issues + Projects**, que el equipo ya usaba para todo el desarrollo. Así evitamos tener los defectos en un sistema y el código en otro, y los PRs cierran sus issues automáticamente.
2. **Behat sin escenarios.** El paso de pruebas de aceptación del plugin corre en el CI pero **no tiene escenarios**: da verde sin probar nada. Lo detectamos en la línea base y queda documentado como riesgo (R9 del plan), pero no lo resolvimos. Es el ejemplo más claro de un “falso verde”.
3. **Brechas de trazabilidad.** De 14 requisitos, 11 están cubiertos y 2 parcialmente (HU-040 y HU-060). **HU-050 (modo multicurso) no tiene test del backend.** Además, el criterio de aceptación de HU-031 quedó desactualizado respecto del código (TRAZABILIDAD.md §3).
4. **Prueba diaria contra el proveedor.** El workflow `integration-external.yml` tiene un `schedule` diario que recién quedó activo el 21/10, cuando promovimos `staging` a `main`: GitHub solo ejecuta los `schedule` desde la rama por defecto. Hasta la entrega acumula pocas corridas diarias.
5. **Carga contra staging.** El plan de la épica decía que la carga iba contra staging. La carga pura se hizo en un backend efímero del pipeline y contra staging solo una corrida chica con el LLM real. Justificación en PRUEBAS_DE_CARGA.md §1.1: medir NexusAI y no al proveedor, no romper el staging que usa el smoke y no gastar dinero.

4. Los ejes del trabajo

Cada eje tiene su documento en `NexusAI/docs/calidad/`. Acá va lo esencial de cada uno.

4.1. Introducción a la calidad

El plan de pruebas fija **objetivos medibles** desde cuatro perspectivas, cada uno con su práctica de proceso. Por ejemplo:

- *alumno*: “el chat responde en menos de 8 s”, medido en TPC-09;
- *equipo*: “cobertura del código nuevo ≥ 80 %”, que hace cumplir el gate;
- *organización*: “costo de tokens dentro del presupuesto”, con el registro de consumo.

El **plan de contingencia** cubre la dependencia más crítica, el proveedor de IA:

- reintentos con backoff;
- una cadena de fallback entre modelos y proveedores;
- un procedimiento para cambiar de proveedor sin tocar código.

Los huecos se reconocen: los timeouts no disparan el fallback y los embeddings no tienen alternativa. Pasaron a la matriz de riesgos y TPC-07 los fijó con tests.

4.2. Fundamentos del testing

- **Arrange–Act–Assert** en todos los tests nuevos, con el **criterio de aceptación escrito en el docstring antes de ejecutar** (PRINCIPIOS_TESTING.md).
- **Defecto real con contraste rojo/verde: DEF-01.** El widget leía la respuesta de Moodle antes de que llegara: la racha y las flashcards mostraban siempre 0.
  1. Primero, el test de regresión en rojo (corrida 38823574292).
  2. Después, el fix en verde (corrida 38823744168).
  3. Además se corrigieron los **37** llamados con ese patrón, no solo los 5 que fallaban, y se agregó un test que impide volver a escribirlo.

  El mismo ciclo se repitió en DEF-07, DEF-08 y DEF-09.
- **Los siete principios, aplicados al proyecto.** Un ejemplo: “las pruebas exhaustivas son imposibles”. Ninguna técnica encontró todos los defectos (sección 6); cada nivel de prueba encontró los suyos.

4.3. Modelos de calidad y niveles de prueba

Matriz ISO/IEC 25010 con las 8 características y su evidencia (PLAN_DE_PRUEBAS.md §7). El **top 3** es:

1. **Seguridad.** Hay datos de alumnos dentro de la plataforma de la universidad, y una fuga entre cursos sería el peor daño posible.
2. **Adecuación funcional.** La IA tiene que responder *solo* con el material del docente y citarlo.
3. **Fiabilidad.** Dependemos de un proveedor externo con cuotas.

Mapa de niveles:

Nivel	Qué se prueba
Unidad	pytest, PHPUnit y Vitest
Integración	Postgres/Redis reales en CI, Moodle real en 7 combinaciones
Integración externa	contratos compartidos entre plugin y backend, y proveedor real
Sistema	smoke en staging y carga
Aceptación	guiones manuales; Behat, pendiente

Despliegue verificado en cada instancia. Cada deploy compara el commit que quedó en la VM con el commit pusheado, y si no coinciden falla (CD_STAGING.md).

4.4. Métricas y análisis estático

El análisis estático lo hacen SonarCloud en cada PR y en `development`, más ruff/mypy, phpcs/phpmd y los auditores de dependencias. Las métricas interpretadas, la de proceso, la de proyecto y las mejoras medidas están en la sección 5 y en METRICAS.md.

4.5. Pruebas dinámicas

64 casos de caja negra sobre el chat, la subida de documentos y el quiz, cada uno con su técnica y su test automatizado (CASOS_DE_PRUEBA.md):

- clases de equivalencia;
- valores límite (n−1, n, n+1);
- **dos tablas de decisión**: la subida con detección de duplicados, de 11 reglas, y el quiz con un tema sin material, de 7 reglas.

Cobertura de caja blanca, antes → después:

Función	Sentencias	Decisiones
`upload_document`	85 % → 100 %	70 % → 100 %
`generate_quiz`	44 % → 100 %	57 % → 100 %

La tabla de decisión del quiz encontró **DEF-08**, un mensaje de error que salía siempre en español. Además apareció algo que vale para el análisis crítico: el esquema del chat tenía 100 % de cobertura de sentencias **sin un solo test de sus límites**. Las reglas de Pydantic son declarativas, así que la cobertura no las "ve".

4.6. Aseguramiento de la calidad

- **Plan de pruebas** versionado (v1.0 a v1.3), con criterios de entrada y salida por nivel y criterio de suspensión.
- **Riesgos** priorizados por probabilidad × impacto. El primero es la caída del proveedor de IA (P×I = 20).
- **QA distinguido de QC.** QA es el flujo de ramas, los PRs, las revisiones, los ADRs, el gate y el plan. QC son las pruebas y los análisis.
- **Plantillas:**

    - caso de prueba (CASOS_DE_PRUEBA.md §1);
    - defecto: formulario de GitHub con pasos, esperado, obtenido y severidad.
- **Trazabilidad** requisito → caso → test → defecto, en las dos direcciones (TRAZABILIDAD.md).

Hallazgo de QA: las plantillas de issue y de PR estaban en una carpeta donde GitHub no las lee, y `development` no tenía protección. Las dos cosas se corrigieron.

4.7. Automatización del testing

- **Pirámide:** la base son los tests unitarios y de componentes (unos 1.160: 621 del backend, 273 del widget y 268 del plugin), después la integración con servicios reales, y arriba el smoke, el E2E y la carga.
- **Pruebas de API e integración externa** (INTEGRACION_EXTERNA.md):
    - contratos compartidos entre plugin y backend: la firma HMAC y las respuestas, que leen los tests de los dos lados;
    - dobles de cada modo de falla del proveedor;
    - pruebas contra Gemini real.
- **Smoke en cada deploy y CI/CD** (CD_STAGING.md). Cada push a `staging`:
    1. despliega por SSH, con una clave restringida a un único comando;
    2. verifica el commit;
    3. corre el E2E.
- **Prueba de regresión por cada defecto corregido** (sección 6).

4.8. Herramientas del ecosistema

- **Mapa de herramientas** por componente y propósito (HERRAMIENTA_SONARCLOUD.md).
- **Herramienta nueva: SonarCloud**, incorporada con los cinco pasos (requisitos, mercado, prueba de concepto, matriz de decisión y despliegue incremental). Incluye lo aprendido usándola, como el conflicto entre una regla de Sonar y el estándar de Moodle.
- **Registro de defectos** en GitHub Issues.
- **Stress y esfuerzo sostenido** con k6 (PRUEBAS_DE_CARGA.md).

5. Evolución de las métricas

5.1. Producto (SonarCloud de `development`, 15/10 → 20/10)

Fuente: análisis de `ded8b7b` (corrida 38752218525) y de `40a97be` (corrida 39438622391). Los valores se verificaron contra el historial de la API de SonarCloud.

Métrica	Antes	Después
Cobertura total del pipeline	56,6 %	**83,6 %**
Cobertura (Sonar: líneas + decisiones)	51,9 %	78,4 %
Cobertura de decisiones (Sonar)	44,7 %	70,4 %
Bugs / confiabilidad	26 / D	**0 / A**
Vulnerabilidades / seguridad	14 / D	**0 / A**
Code smells / mantenibilidad	831 / A	797 / A
Deuda técnica	4.676 min	4.600 min
Duplicación	2,5 %	2,4 %
Complejidad ciclomática cada 1.000 líneas	135,6	135,6
Líneas de código	35.990	36.093

De las 14 vulnerabilidades, 13 se corrigieron en el código. La restante (un `sha1` que agrupa preguntas repetidas, sin uso de seguridad) se revisó y se aceptó en SonarCloud, con la justificación registrada en el propio hallazgo.

5.2. Mejoras medidas antes/después

1. **Cobertura: 56,6 % → 83,6 %** (TPC-05).

Componente	Antes	Después
Plugin	41,8 %	85,5 %
Widget	40,0 %	78,9 %
Backend	84,1 %	84,2 %

2. **Confiabilidad y seguridad: D → A** (#646). Entre los hallazgos había **3 bugs reales**: una tarea de indexación que se podía perder, errores de red sin manejar en el widget y un `assert` que el propio código ignoraba.
3. **Capacidad bajo carga** (DEF-09):

	Antes	Después
Chats OK en el stress	68 %	99,95 %
p95 con 100 usuarios	60 s	2,55 s
Throughput con 100 usuarios	colapso	19,4 pedidos/s
Registros de costo perdidos	522	0

5.3. Métrica de proceso: lead time de los PRs

Período	PRs	Mediana	p90	Media
Antes del TP (24/07 – 12/10)	132	0,2 h	39,9 h	15,5 h
Durante el TP (13/10 – 20/10)	57	0,5 h	10,5 h	3,3 h

La mejora está en la cola, no en la mediana: los PRs chicos, de un solo tema y con el CI en verde dejaron de esperar días. La tasa de fallos del pipeline en `development` subió de 9,8 % a 18,4 %, pero **las 14 fallas son problemas reales**:

- 13 por un token vencido (DEF-05);
- 1 por el proveedor gratuito, que tardó 112 s en contestar.

Ver METRICAS.md §3.

5.4. Métrica de proyecto: avance y velocidad

Al 20/10 estaban hechos **30 de 35 story points (86 %)**:

- velocidad de **3,75 SP/día**, contra 2,5 planificados;
- 7 de 9 subtareas cerradas antes de su fecha.

La velocidad usa nuestra propia estimación, así que también mide qué tan bien estimamos: TPC-03, TPC-05 y TPC-06 estaban sobreestimadas.

5.5. Desempeño

SLA (TESTING.md §6)	Meta	Medición	Estado
Consulta RAG, p95	< 8 s	4,39 s en staging con el LLM real; 2,52 s con 100 usuarios simulados	Cumplido
Re-subida duplicada, p95	< 1 s	0,03 s	Cumplido
Arranque en frío	< 15 s	11,61 s	Cumplido

5.6. Ensayo de la demo (21/10)

Después de promover `staging` a `main` ensayamos los pasos de la demo contra el ambiente real:

Paso	Corrida	Resultado
Deploy + smoke	39665714741	Intento 1 rojo: un contenedor viejo en la VM impidió el recreate y la verificación del commit lo detectó. Intento 2 verde, con el commit verificado y el smoke OK
Integración externa	39681910580	Rojo: el chat contra Gemini tardó 33,15 s (SLA 8 s). Fallback y embeddings en verde
Carga contra staging	39681914662	Verde: p95 de 5,03 s con el LLM real

Las dos corridas rojas son el pipeline haciendo su trabajo: una detectó estado sucio en la infraestructura y la otra, que el proveedor no cumple nuestro SLA.

6. Defectos

ID	Defecto	Severidad	Lo encontró	Prueba de regresión
DEF-01	La racha y las flashcards mostraban siempre 0 (se leía la respuesta de Moodle antes de que llegara)	Media	Tests unitarios	`ajaxUsage.test.js`
DEF-02	Toda pregunta por voz se rechazaba	Alta	Uso real	Tests en las tres capas
DEF-03	El pedido de datos personales (GDPR) no exportaba ni borraba nada	Alta	Primera corrida de PHPUnit	`privacy_provider_test.php`
DEF-04	Al borrar un curso no se purgaban los posts de foro indexados	Alta	Auditoría del Marketplace	`backend_tasks_test.php`
DEF-05	El pipeline de publicación fallaba por un token vencido	Media	**El pipeline**	La corrida del workflow
DEF-06	Un doble de prueba sobrevivía entre tests	Baja	El pipeline	`external_testcase.php`
DEF-07	La indexación en lote con Gemini fallaba	Media	**Proveedor real**	`test_providers.py`
DEF-08	Un rechazo del quiz salía siempre en español	Baja	**Tabla de decisión**	`test_quiz_router.py`
DEF-09	El chat colapsaba con 40 alumnos simultáneos	Alta	**Prueba de stress**	`test_db_pool_contention.py`

ID	Defecto	Severidad	Lo encontró	Prueba de regresión
DEF-10	Vulnerabilidades nuevas en dos dependencias (Pillow, vite)	Media	**Pipeline** (auditorías)	`pip-audit`, `npm audit` en cada PR

Ejecuciones fallidas que detectaron un problema real:

- **DEF-05:** el workflow que publica los repos derivados falló 13 veces el 14/10 por un token vencido. Sin el pipeline, el repo del Marketplace se habría quedado desactualizado sin que nadie lo notara.
- **DEF-10:** el 21/10, el PR que llevaba `development` a `staging` dio rojo en las auditorías de dependencias (backend, widget) sin que cambiara nuestro código. Se habían publicado 8 CVE de Pillow y 3 vulnerabilidades *high* de vite. Se corrigió antes de llegar a staging.

Cada defecto tiene sus pasos, esperado, obtenido, la causa raíz (con los 5 porqués en DEF-01 y DEF-09) y las corridas del rojo y el verde en REGISTRO_DEFECTOS.md.

7. Análisis crítico

7.1. Lo que funcionó

- **Combinar técnicas.** Cada técnica encontró lo que las otras no podían ver:
  - los dobles de prueba no mostraban que Gemini incumple una parte del contrato de OpenAI (DEF-07);
  - las pruebas funcionales no mostraban el interbloqueo bajo concurrencia (DEF-09);
  - la cobertura no mostraba los límites sin probar.

  La mayor lección del TP es que **ningún tipo de prueba alcanza solo**.
- **El quality gate como freno, no como meta.** Con la protección de `development`, ningún PR entró con cobertura de código nuevo menor al 80 %. Los PRs del TP pasaron todos por ahí, incluidos los que agregaron tests a otros.
- **El despliegue automático con verificación.** Hoy un deploy a staging tarda minutos y se prueba solo. Antes era manual, y el que verificaba era el admin de deploy.

7.2. Lo que aprendimos de lo que no salió bien

- **El gate no mejora lo heredado.** Los 26 bugs y las 14 vulnerabilidades no se movieron en cinco días de PRs, porque el gate solo mira el código nuevo. Bajaron recién cuando los atacamos a propósito. Un gate mantiene la calidad, pero no la sube.
- **Los verdes que no prueban nada.** Behat con 0 escenarios, un umbral de k6 sin muestras y una prueba diaria que nunca se programó. Las tres cosas daban la impresión de que se estaba probando algo que no se probaba. Ahora el k6 exige que haya chats enviados, pero Behat sigue pendiente.
- **Errores de configuración que parecen caídas.** Reportamos staging como caído cuando el problema era una URL copiada de documentación desactualizada. Desde entonces, la URL vive en una variable del pipeline.
- **El proveedor gratuito no es confiable en latencia.** Picos de 20 s y de 112 s en el plan gratis de Gemini. Además, Google retiró el modelo de producción para proyectos nuevos sin avisar. La integración real tiene que probarse seguido justamente por eso.
- **Un pipeline verde puede ponerse rojo sin tocar el código.** El 21/10 las auditorías frenaron el deploy de la demo por vulnerabilidades publicadas ese día (DEF-10). Las dependencias también cambian, y por eso las auditorías tienen que correr en cada PR y no solo al principio.
- **Las métricas necesitan contexto.** La tasa de fallos del pipeline “empeoró” porque el pipeline detectó más problemas reales. La velocidad del proyecto “mejoró” en parte porque sobreestimamos algunas tareas.

7.3. Riesgos que quedan abiertos

Riesgo	Recomendación
Producción usa un modelo de Gemini que Google ya no ofrece a proyectos nuevos	Migrar `LLM_MODEL` antes de rotar la key
Un solo proceso uvicorn, VM de 1 GB	Medir el techo de la VM; pasar a 2 workers si crece el uso
La moderación usa una llamada al LLM por pregunta	Configurar `MODERATION_API_KEY` (gratis y más rápida)
Quiz y resumen retienen la conexión a la base durante el LLM	Extender el patrón de DEF-09
Behat sin escenarios; HU-050 sin test del backend	Escenarios de aceptación y test del multicurso
Gemini supera el SLA de 8 s de forma recurrente (112 s, 33 s y 65 s en corridas distintas)	Seguirlo con la corrida diaria; evaluar un timeout más corto con fallback a otro modelo

7.4. Qué haríamos distinto

- **Correr la carga antes.** DEF-09 estuvo en producción todo el tiempo y solo apareció con concurrencia.
- **Atacar la deuda heredada desde el principio**, no solo frenar la nueva.
- **Configurar el `schedule` en `main` desde el primer día** y verificar que efectivamente corra.

8. Conclusiones

El proceso de calidad dejó de ser una lista de herramientas y pasó a ser parte de cómo trabajamos: hoy cada cambio de NexusAI pasa por pruebas en tres lenguajes, análisis estático con un gate que bloquea, despliegue automático a staging y una prueba de punta a punta contra el proveedor real. Las metas del nivel 3 se cumplen, y los números lo muestran: 83,6 % de cobertura, A/A/A en SonarCloud y capacidad para 100 alumnos simultáneos dentro del SLA.

Lo más valioso del TP no fueron las métricas, sino los **10 defectos reales** que encontró. Varios eran graves y ninguno se había detectado antes:

- el pedido de datos personales que no hacía nada;
- el colapso bajo carga;
- los registros de costo que se perdían.

Cada uno quedó corregido con una prueba que impide que vuelva. Y lo que no cumplimos (Behat, una brecha de trazabilidad, la prueba diaria) está identificado, medido y con su próximo paso.

9. Aporte individual

Armado con la evidencia del repositorio (commits, PRs e issues) y confirmado por el equipo.

Integrante	Aporte en el TP
Delfina Salinas	Coordinación de la épica TPC. Línea base, SonarCloud y quality gate, plan de pruebas, casos de caja negra y trazabilidad, tests unitarios y cobertura, registro de defectos y corrección de DEF-01 y DEF-06 a DEF-10, integración externa, CD a staging, pruebas de carga y métricas (TPC-01 a TPC-10)
Santiago Tricherri	Administración del despliegue en Oracle. Configuración del CD (clave restringida, environment `staging`, bootstrap), corrección de DEF-02 (preguntas por voz, #591), release 0.19.0 y build de los módulos AMD, modelo de producción

Integrante	Aporte en el TP
Marcos Bugliotti	Corrección de DEF-04 (purga de posts de foro al borrar un curso, #589), paso Grunt estricto en el CI del plugin (#593, #594), arreglos del feed de calendario y de la subida de archivos

Las once subtareas tienen responsable “todos”: cada integrante tiene que poder explicar cualquier parte del trabajo, como pide el enunciado.

10. Uso de inteligencia artificial

Usamos **Claude Code** (Anthropic) como asistente durante el TP, para escribir código de pruebas, scripts (k6, deploy, análisis), fixes y borradores de documentación. El enunciado lo permite si el grupo entiende y puede explicar todo lo entregado. Para eso:

- todo cambio entró por un **PR revisado** y con el **CI en verde**: la IA no mergeó ni desplegó nada;
- las decisiones las tomó el equipo: el nivel, el alcance de cada tarea, qué corregir y qué aceptar;
- **los números salen de las corridas** del pipeline y de SonarCloud, nunca de texto generado;
- cuando la IA se equivocó, quedó registrado. Por ejemplo, la falsa alarma de staging, documentada en INTEGRACION_EXTERNA.md.

11. Anexo A: entregables del enunciado

Entregable	Dónde está
Plan de pruebas versionado, con matriz ISO 25010 y trazabilidad	`docs/calidad/PLAN_DE_PRUEBAS.md`, `TRAZABILIDAD.md`
Código de las pruebas (unitarias, integración, smoke, stress, esfuerzo), ejecutables con un comando	`services/api/tests` (`pytest`), `plugin/local/nexusai/tests` (`moodle-plugin-ci phpunit`), `react/src/**/*.test.*` (`npx vitest run`), `scripts/e2e_deployed.py`, `scripts/load/run_load.sh`
Pipeline de CI como archivo, con una corrida completa en verde y una fallida que detectó un problema real	`.github/workflows/`. Verde: cualquier PR del TP, por ejemplo #646 (18/18). Fallidas reales: DEF-05 y DEF-10
Reportes automáticos de cobertura, análisis estático y stress	Cobertura y SonarCloud en cada PR (`quality-gate.yml`); stress en el artifact de `load-test.yml`
Documentación de la integración externa	`docs/calidad/INTEGRACION_EXTERNA.md`, `NexusAI/contracts/`
Registro de defectos completo	GitHub Issues (label `tipo:defecto`) y `docs/calidad/REGISTRO_DEFECTOS.md`
Documento de la herramienta y matriz de decisión	`docs/calidad/HERRAMIENTA_SONARCLOUD.md`
Informe PDF	Este documento (`docs/calidad/informe/`)
Demostración con un despliegue nuevo	Guion en `docs/calidad/DEMO.md`

12. Anexo B: índice de documentos

Documento	Subtarea
`EPICA_TP_CALIDAD.md`: plan del TP	TPC-00
`LINEA_BASE.md`: medición inicial y nivel	TPC-01
`HERRAMIENTA_SONARCLOUD.md`: herramienta nueva	TPC-02
`PLAN_DE_PRUEBAS.md`: plan, riesgos, ISO 25010, QA/QC	TPC-03
`CASOS_DE_PRUEBA.md` y `TRAZABILIDAD.md`	TPC-04
`PRINCIPIOS_TESTING.md`: AAA, cobertura, 7 principios	TPC-05
`REGISTRO_DEFECTOS.md`	TPC-06
`INTEGRACION_EXTERNA.md`	TPC-07
`CD_STAGING.md`	TPC-08
`PRUEBAS_DE_CARGA.md`	TPC-09
`METRICAS.md`	TPC-10
`DEMO.md` y este informe	TPC-11